



实验一：PyTorch快速入门

《深度学习》实验课
2026年 春

深度学习——实验课程安排

本学期实验课程共8个学时，4 个实验项目，总成绩为 100 分。

实验项目	一	二	三	四
学时	2	2	2	2
实验内容	PyTorch快速入门	正则化技术实践	模型优化实践	神经网络项目实践
分数	10	20	20	50
上课时间 (地点)	第10周 周三 (T2102)	第11周 周三 (T2102)	第13周 周三 (T2102)	第16周 周三 (T2102)
检查方式	提交实验报告、工程文件			

目录

1	主流深度学习框架介绍
2	PyTorch环境搭建
3	PyTorch实例应用
4	实验作业

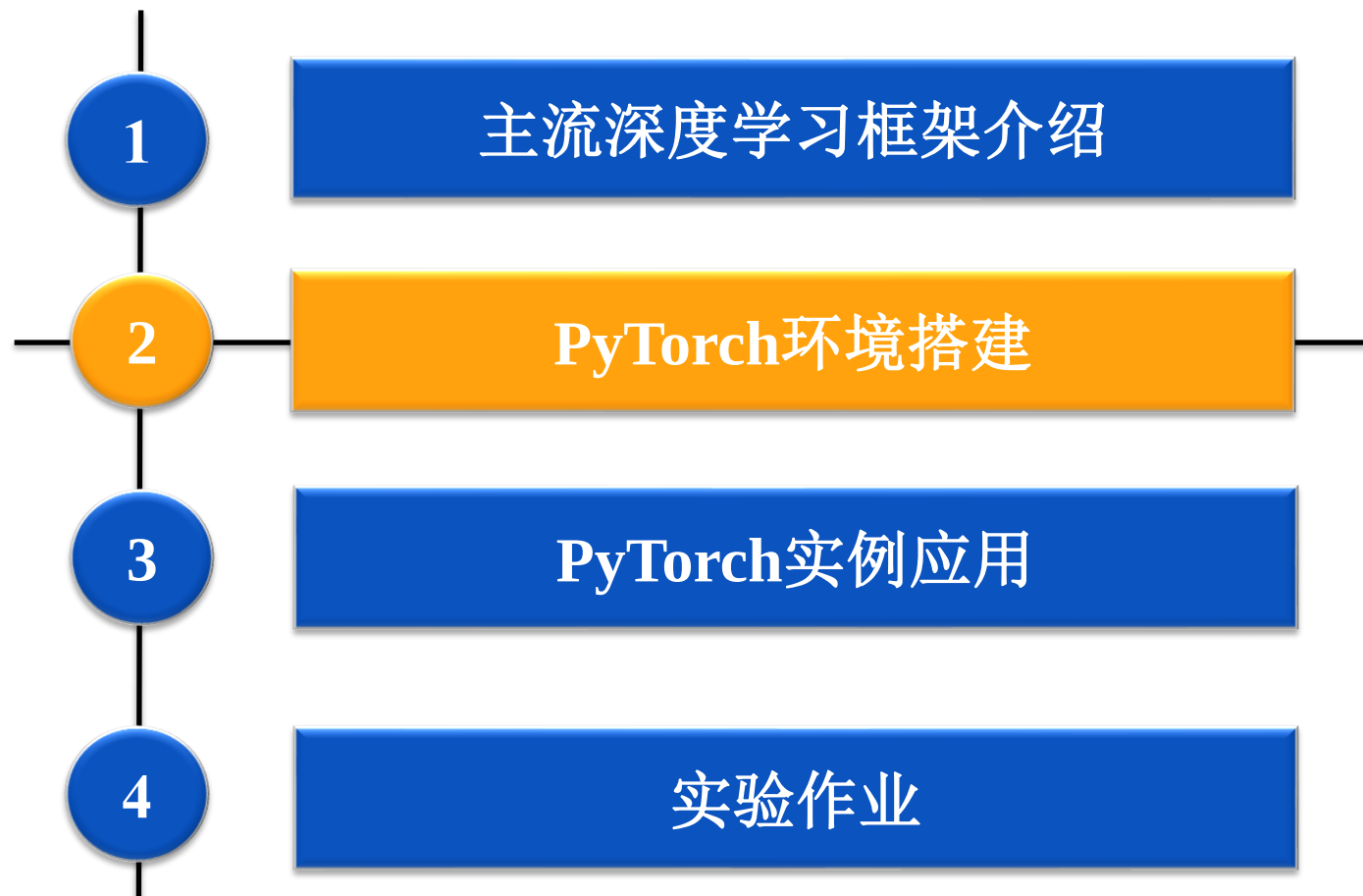
主流深度学习框架介绍

➤ 随着深度学习算法的发展，出现了许多深度学习框架，这些框架各有所长，各具特色。常用开源框架就有 **PyTorch**、Keras、Caffe、**TensorFlow**、Theano、CNTK、MXNet、**PaddlePaddle**、**MindSpore**等。

框架	主导方	核心哲学与特点	易用性
PyTorch	Meta (Facebook)	研究优先，灵活直观。以动态图（Eager Execution）起家，拥有非常Pythonic的API设计，调试简单， 是学术界和研究中最受欢迎的框架。	☆☆☆☆☆ (最容易)
TensorFlow	Google	生产优先，全面强大。以静态图起家，拥有强大的生产端部署工具链（TFX, TFLite, TF.js等），生态极其庞大， 是企业级应用的传统强者。	☆☆☆ (中等)
PaddlePaddle	百度	产业实践，中文友好。强调与产业结合，内置了众多经过实践验证的模型库（如PaddleOCR、PaddleClas）。 中文文档和社区支持是其最大优势。	☆☆☆☆ (对中文用户友好)
MindSpore	华为	全场景协同，性能优化。主打“端-边-云”全场景协同部署，与华为昇腾AI处理器（Ascend）深度绑定，追求极致的性能。	☆☆☆ (中等偏上)

- 不管是哪种深度学习框架，都能在CPU和 NVIDIA GPU（通过CUDA）资源上应用。
- 小规模数据训练可在 CPU 上完成，适合学习开发。
- 大规模模型训练：GPU 是必选项，极大提升效率。

目录



环境准备

- 先准备一个python环境，版本在**Python 3.8–3.11**，另外各个操作系统的版本要求如下：
 - **Windows 10及以上 / Ubuntu 16.04 及以上 / macOS 10.12.6 及以上**
- 注意安装**PyTorch**时，需要根据电脑是否具有相应GPU，来确定是安装性能较强的GPU版本或CPU版本。

我们选择在Windows10系统环境下来配置深度学习做示例：

具体硬件配置（笔记本）

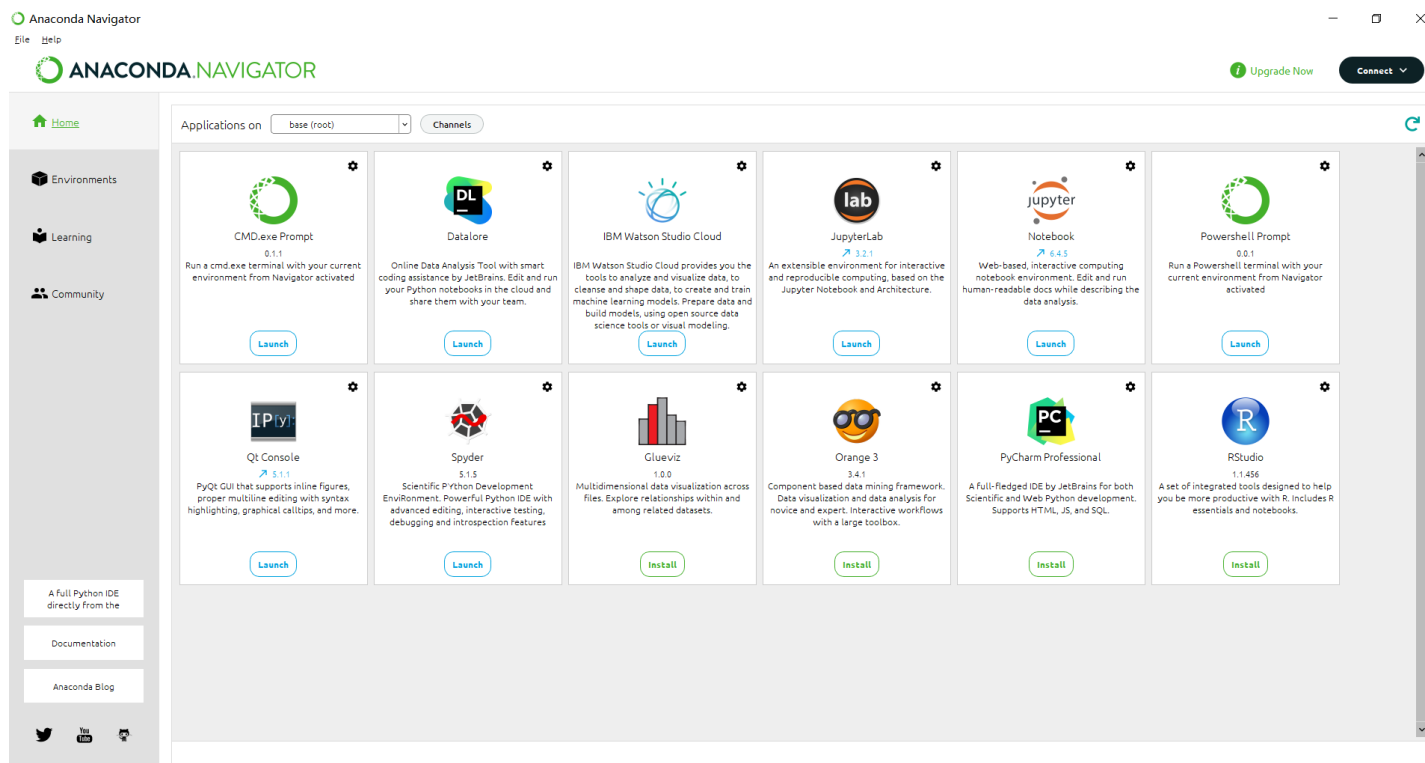
- 处理器：Intel (R) Core (TM) i7
- 内存：8GB以上
- 硬盘：SSD
- GPU：NVIDIA GeForce GTX 960（非必须）

软件环境

- 只需要一般配置的计算机就能正常安装PyTorch的CPU版本。
- 但如果想要成功安装GPU版本，除了具备GPU的硬件资源外，还需要从NVIDIA官方网站下载及安装CUDA和cuDNN这两个软件。

环境准备——Anaconda

- Anaconda是一个非常流行的基于Python的科学计算环境，里面包含了python、conda(一个python包管理器)和各种用于科学计算的包，可以完全独立使用，不再额外下载Python。
- Anaconda官网: <https://www.anaconda.com/>
- 安装完毕后，运行Anaconda Navigator得到界面如图所示。



环境准备——Jupyter Notebook

- Jupyter Notebook是Anaconda默认提供的一款交互式的开发环境，该环境即可以集成Python，同时也可以集成R语言。Jupyter Notebook是以网页的形式打开，可以在网页页面中直接编写代码和运行代码，代码的运行结果也会直接在代码块下显示。



环境准备——PyTorch环境搭建

❖ (1) 环境要求

- Python: 3.8 ~ 3.11
- 系统: Windows10+/Ubuntu16.04+/macOS10.12.6+
- 硬件: 内存 $\geq$ 8GB, SSD, NVIDIA GPU 可选

❖ (2) 工具: Anaconda+ Jupyter Notebook

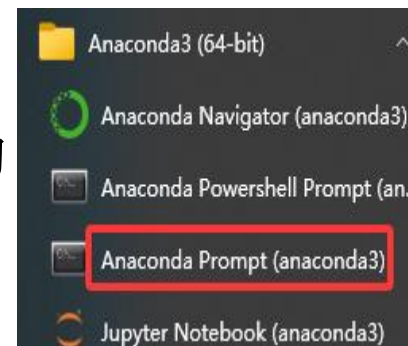
- 统一管理 Python 环境, 避免版本冲突 ;
- Jupyter 交互式编写运行代码, 适合实验教学。



环境准备——PyTorch环境搭建

❖ (3) 搭建PyTorch CPU环境

- 打开Anaconda Prompt终端，执行如下conda命令，用来创建一个干净的python环境(推荐，避免跟其他python版本有冲突)。



##创建一个新3.8版本的python环境，名字为 *pytorch*

```
conda create -n pytorch python=3.8
```

##然后用以下命令来激活python环境

```
conda activate pytorch
```

##验证环境是否创建成功，执行命令

```
python -V
```

```
Anaconda Prompt (anaconda3)

(base) C:\Users\lenovo>conda create -n pytorch python=3.8
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_rep
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
(pytorch) C:\Users\lenovo>python -V
Python 3.8.20
```

注意：新python版本在 3.8-3.11 均可以。

环境准备——PyTorch环境搭建

❖ (3) 搭建PyTorch CPU环境

- 使用pip命令 安装PyTorch CPU

`pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu`

```
(base) C:\Users\lenovo>pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
Keyring is skipped due to an exception: 'keyring.backends'
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch
  Downloading https://download.pytorch.org/whl/cpu/torch-1.13.1%2Bcpu-cp37-cp37m-win_amd64.whl (164.2 MB)
    | 164.2 MB 283 kB/s
Collecting torchvision
  Downloading https://download.pytorch.org/whl/cpu/torchvision-0.14.1%2Bcpu-cp37-cp37m-win_amd64.whl (1.1 MB)
    | 1.1 MB 1.6 MB/s
Collecting torchaudio
  Downloading https://download.pytorch.org/whl/cpu/torchaudio-0.13.1%2Bcpu-cp37-cp37m-win_amd64.whl (2.0 MB)
    | 2.0 MB 1.3 MB/s
Requirement already satisfied: typing-extensions in c:\users\lenovo\appdata\roaming\python\python37\site-packages (from
torch) (4.5.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in e:\anaconda3\lib\site-packages (from torchvision) (7.0.0)
```

- 验证其否安装成功，执行命令 `pip show torch`

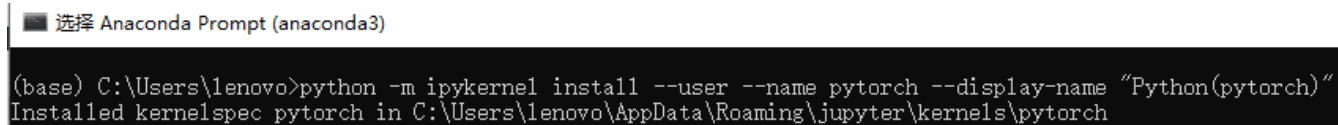
```
(pytorch) C:\Users\lenovo>pip show torch
Name: torch
Version: 1.12.1
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
Home-page: https://pytorch.org/
Author: PyTorch Team
Author-email: packages@pytorch.org
License: BSD-3
Location: c:\programdata\anaconda3\envs\pytorch\lib\site-packages
Requires: typing_extensions
Required-by: torchaudio, torchvision
```

环境准备——PyTorch环境搭建

🧩 (3) 搭建PyTorch CPU环境

➤ **Jupyter内核配置：** 打开Anaconda Prompt终端，输入如下命令

```
python -m ipykernel install --user --name pytorch --display-name "Python(pytorch)"
```



```
选择 Anaconda Prompt (anaconda3)  
(base) C:\Users\lenovo>python -m ipykernel install --user --name pytorch --display-name "Python(pytorch)"  
Installed kernelspec pytorch in C:\Users\lenovo\AppData\Roaming\jupyter\kernels\pytorch
```

然后打开Jupyter，就可以从new->Python（pytorch）选择新安装的名叫pytorch的环境，即可以开始编程之路。



(3) 搭建PyTorch CPU环境

- 其他（环境管理），在Anaconda Prompt终端中执行以下命令

查看所有环境: `conda env list`

退出环境: `conda deactivate`

删除环境: `conda env remove -n pytorch` (一般不操作!!)

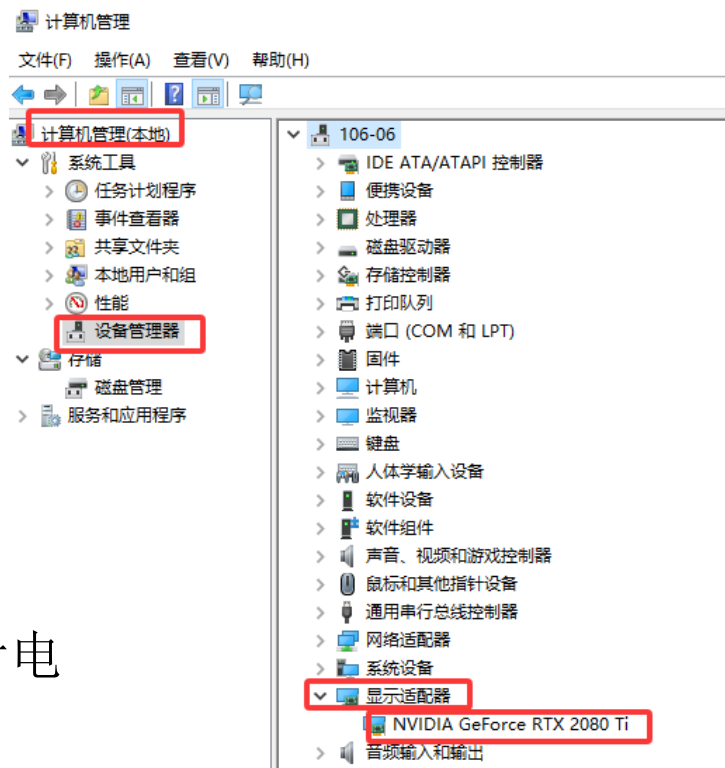
环境准备——PyTorch环境搭建

❖ (4) 搭建PyTorch GPU环境（可选）

➤ 安装PyTorch GPU环境最重要的三个依赖：

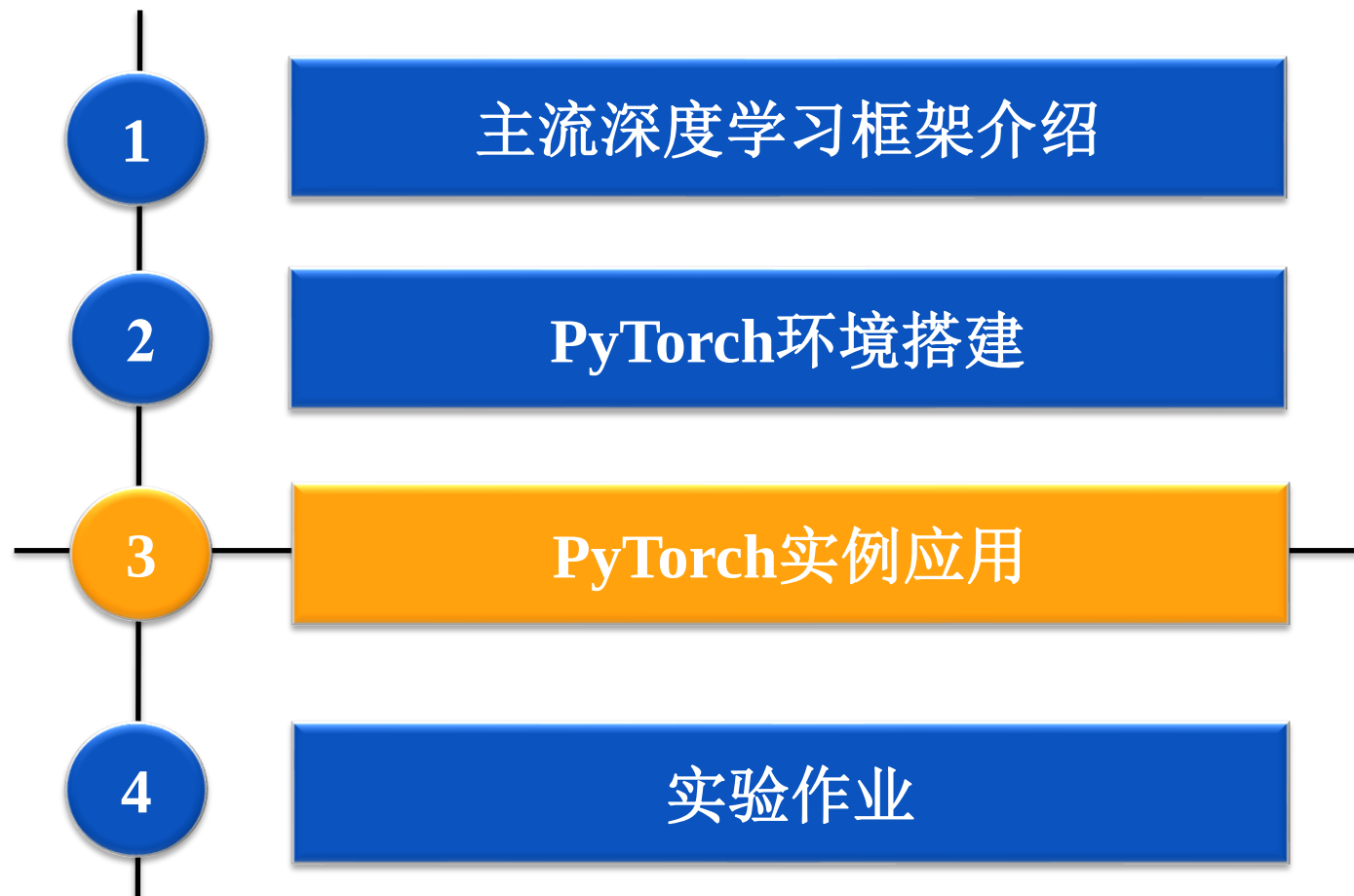
- ① NVIDIA 显卡驱动：版本要足够新，以支持你所需的 CUDA 版本。
- ② CUDA工具包：NVIDIA 的并行计算平台。
- ③ cuDNN SDK：NVIDIA 的深度神经网络加速库。

➤ 安装PyTorch GPU版本时，首先**检查电脑是否有GPU显卡**，如查看电脑显卡情况：右键**此电脑**→**管理**→**设备管理器**→**显示适配器**



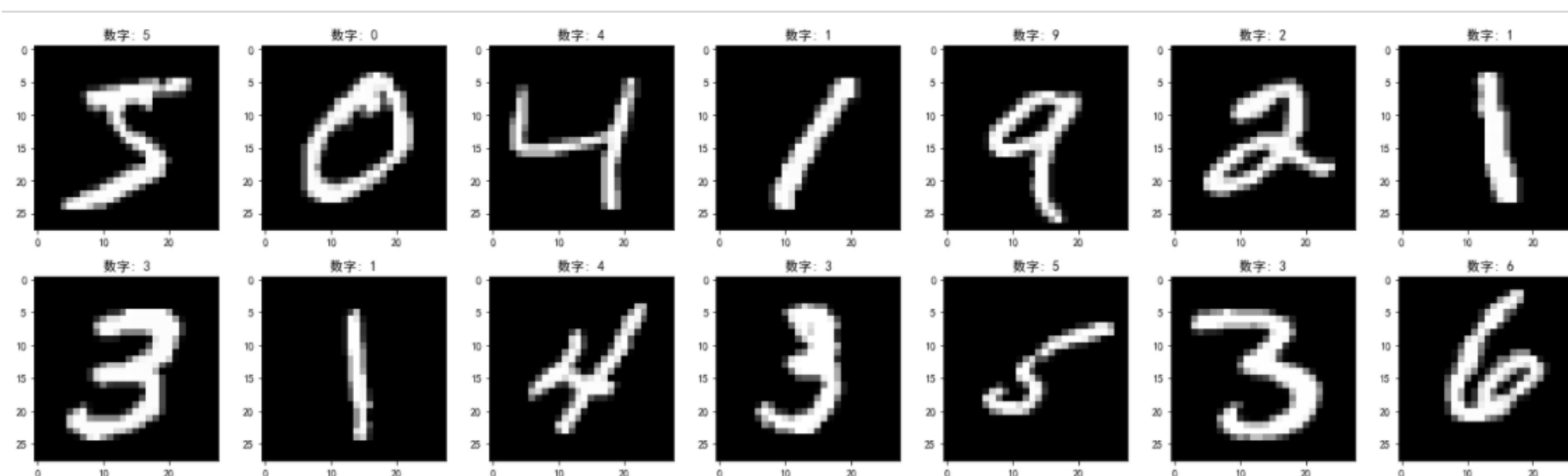
如上图所示，有 NVIDIA 显卡，可自行安装 GPU 版 PyTorch(请自行安装)。

目录



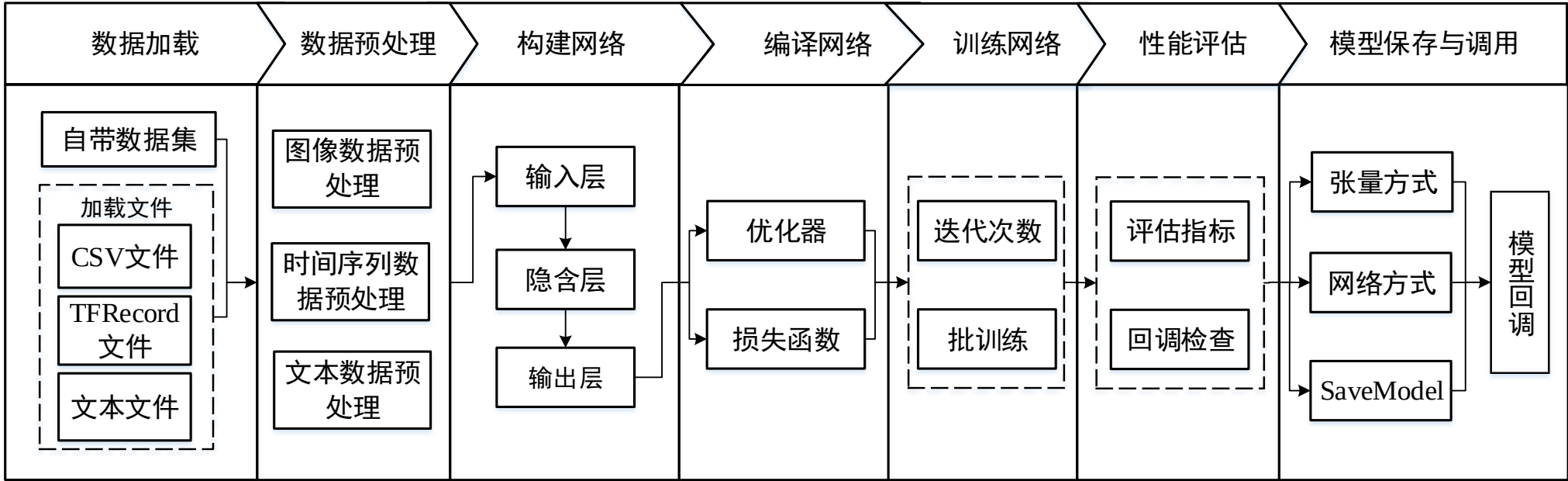
MNIST数据集概述

- MNIST（全称为 Modified National Institute of Standards and Technology）手写数字识别数据集是由Yann LeCun所收集的，他也是Convolution Neural Network的创始人。MNIST数字文字识别数据集数据量不会太多，而且是单色的图像，比较简单、很适合深度学习的初学者用来练习建立模型、训练、预测。
- **MNIST数据集共有训练数据60000项、测试数据10000项。** MNIST数据集中的每一项数据都由images（数字图像）和labels（真实的数字）所组成。MNIST数据集可通过`torchvision.datasets.MNIST`加载，其中包含4个Numpy数组。



PyTorch深度学习通用流程

- 深度学习通常包含数据加载与预处理、构建网络、训练网络和性能评估几个主要的步骤。
- PyTorch为每个步骤提供了一些相应的函数，使得可以方便快速地进行深度学习。
- 本章以手写数字识别的例子介绍PyTorch深度学习的通用流程。
- 深度学习的通用流程如图所示，分为以下步骤：



运行实例

PyTorch搭建神经网络：

- 第一步：导入相关模块，如`import torch`。
- 第二步：设置设备，检查是否有可用的GPU，如果有则使用GPU，否则使用CPU
- 第三步：数据加载和预处理，如使用`torchvision.datasets.MNIST`加载MNIST数据集，使用`transforms`进行数据预处理和标准化。

```
# 导入必要的库
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 设置设备 - 检查是否有可用的GPU，如果有则使用GPU，否则使用CPU
# 这可以显著加速训练过程，因为GPU擅长并行计算
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"使用设备: {device}") # 打印当前使用的设备信息
```

```
# 数据预处理和加载
# transforms.Compose 将多个数据转换操作组合在一起
transform = transforms.Compose([
    transforms.ToTensor(), # 将PIL图像或numpy数组转换为PyTorch张量
    transforms.Normalize((0.1307,), (0.3081,)) # 使用MNIST数据集的均值和标准差进行标准化
    # 标准化公式: (input - mean) / std
])

# 加载MNIST数据集
# datasets.MNIST是PyTorch提供的MNIST数据集加载器
# root: 数据集存储路径
# train=True: 加载训练集
# download=True: 如果数据集不存在则自动下载
# transform: 应用上面定义的数据转换
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)

# 创建数据加载器
# DataLoader负责批量加载数据，支持多进程数据加载和数据打乱
# batch_size=64: 每个批次包含64个样本
# shuffle=True: 在每个epoch开始时打乱训练数据顺序
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False) # 测试集不需要打乱
```

运行实例

PyTorch搭建神经网络：

- 第四步：逐层搭建网络结构，如 `model = nn.Sequential()`。
- 第五步：配置训练方法，包含优化器、损失函数。
- 第六步：执行训练循环过程，如使用 `model.train()` 设置模型为训练模式。

```
# 建立全连接神经网络模型
# nn.Sequential是一个顺序容器，模块将按照它们在构造函数中传递的顺序添加
model = nn.Sequential(
    nn.Flatten(), # 将28x28的图像展平为784维的向量
    nn.Linear(784, 128), # 全连接层，输入维度784，输出维度128
    nn.ReLU(), # ReLU激活函数，引入非线性
    nn.Linear(128, 10) # 输出层，输入维度128，输出维度10（对应10个数字类别）
).to(device) # 将模型移动到指定的设备（GPU或CPU）

# 配置训练方法：优化器、损失函数
# optim.Adam是一种自适应学习率的优化算法，结合了AdaGrad和RMSProp的优点
optimizer = optim.Adam(model.parameters()) # 传入模型参数以便优化器知道要优化什么
# nn.CrossEntropyLoss是交叉熵损失函数，适用于多分类问题
# 它内部已经包含了Softmax操作，所以模型最后一层不需要添加Softmax
criterion = nn.CrossEntropyLoss()
```

```
# 训练模型
model.train() # 将模型设置为训练模式，这会启用dropout和batch normalization等训练特定行为
for epoch in range(5): # 训练5个epoch
    for data, target in train_loader: # 遍历训练数据加载器中的每个批次
        data, target = data.to(device), target.to(device) # 将数据移动到指定设备

        optimizer.zero_grad() # 清除之前的梯度，防止梯度累积
        output = model(data) # 前向传播，计算模型输出
        loss = criterion(output, target) # 计算损失
        loss.backward() # 反向传播，计算梯度
        optimizer.step() # 更新模型参数

    print(f'Epoch {epoch+1} completed') # 打印当前epoch完成信息
```

运行实例

PyTorch搭建神经网络：

- 第七步：使用`model.eval()`做模型评估，返回测试集上的损失和准确率。

```
# 模型评估
model.eval() # 将模型设置为评估模式，这会禁用dropout和batch normalization等训练特定行为
test_loss = 0 # 初始化测试损失
correct = 0 # 初始化正确预测的样本数
total = 0 # 初始化总样本数

with torch.no_grad(): # 禁用梯度计算，节省内存和计算资源
    for data, target in test_loader: # 遍历测试数据加载器中的每个批次
        data, target = data.to(device), target.to(device) # 将数据移动到指定设备
        output = model(data) # 前向传播，计算模型输出
        test_loss += criterion(output, target).item() # 累加批次损失
        pred = output.argmax(dim=1, keepdim=True) # 获取预测结果（概率最大的类别）
        correct += pred.eq(target.view_as(pred)).sum().item() # 统计正确预测的数量
        total += target.size(0) # 累加样本数量

# 计算平均测试损失和准确率
test_loss /= len(test_loader) # 平均损失 = 总损失 / 批次数量
accuracy = 100. * correct / total # 准确率 = 正确预测数 / 总样本数 * 100%

# 打印测试结果
print(f'Test set: Average loss: {test_loss:.4f}, Accuracy: {correct}/{total} ({accuracy:.2f}%)')
```

在GPU环境的运行结果如下：

```
使用设备: cuda
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Test set: Average loss: 0.0788, Accuracy: 9752/10000 (97.52%)
```

在CPU环境的运行结果如下：

```
使用设备: cpu
Epoch 1 completed
Epoch 2 completed
Epoch 3 completed
Epoch 4 completed
Epoch 5 completed
Test set: Average loss: 0.0801, Accuracy: 9752/10000 (97.52%)
```

PyTorch函数用法和介绍——搭建网络结构（1）

➤ nn.Sequential()

- PyTorch中的nn.Sequential是一个**顺序容器**，用于按顺序包装多个网络层。在构建神经网络时，我们可以使用nn.Sequential将一系列的层组合在一起，这样数据就会按照我们定义的顺序依次通过这些层。基本语法如下：

```
model = nn.Sequential(  
    layer1,  
    layer2,  
    layer3,  
    ...  
)
```

- 有三种创建方式：

1. 直接传递层实例

```
import torch.nn as nn  
  
model = nn.Sequential(  
    nn.Linear(784, 256),  
    nn.ReLU(),  
    nn.Linear(256, 10),  
    nn.Softmax(dim=1)  
)
```

2. 使用 OrderedDict（有序字典）

```
from collections import OrderedDict  
  
model = nn.Sequential(OrderedDict([  
    ('flatten', nn.Flatten()),  
    ('fc1', nn.Linear(784, 256)),  
    ('relu', nn.ReLU()),  
    ('fc2', nn.Linear(256, 10)),  
    ('softmax', nn.Softmax(dim=1))  
]))
```

3. 使用 add_module 方法

```
model = nn.Sequential()  
model.add_module('flatten', nn.Flatten())  
model.add_module('fc1', nn.Linear(784, 256))  
model.add_module('relu', nn.ReLU())  
model.add_module('fc2', nn.Linear(256, 10))  
model.add_module('softmax', nn.Softmax(dim=1))
```


PyTorch函数用法和介绍——搭建网络结构（2）

➤ **nn.Sequential()**

Sequential函数与常用的神经网络结构层以及激活函数一起搭配使用，如：

1) 全连接层: **nn.Linear(in_features, out_features)**

2) 卷积层: **nn.Conv2d(in_channels, out_channels, kernel_size)**

3) 池化层: **nn.MaxPool2d(kernel_size)** # 最大池化层

nn.AvgPool2d(kernel_size) # 平均池化层

4) 正则化层: **nn.Dropout(p=0.5)** # Dropout 层

nn.BatchNorm2d(num_features) # 批量归一化层

5) 其他实用层: **nn.Flatten()** # 展平层，将多维输入展平为一维

nn.Unflatten(dim, unflattened_size) # 反展平层

6) 激活函数:

nn.ReLU() # ReLU 激活函数

nn.Sigmoid() # Sigmoid 激活函数

nn.Tanh() # Tanh 激活函数

nn.Softmax(dim) # Softmax 激活函数

PyTorch函数用法和介绍——配置训练方法1

➤ 在 PyTorch 中，配置训练方法主要涉及选择和使用优化器（Optimizer）以及损失函数（Loss Function）。这两个组件是训练神经网络的核心部分，直接影响模型的收敛速度和最终性能。

1) **优化器 (Optimizers)**，负责根据损失函数的梯度更新模型的参数，以最小化损失函数，常用优化器介绍，**但不限于此**。

优化器	使用方法
SGD	<pre>import torch.optim as optim optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)</pre> # 参数说明: # - model.parameters(): 需要优化的模型参数 # - lr: 学习率 (learning rate) # - momentum: 动量系数, 加速收敛并减少震荡
Adam	<pre>optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999), eps=1e-08)</pre> # - betas: 用于计算梯度及其平方的移动平均的系数 # - eps: 数值稳定性项, 防止除以零
RMSprop	<pre>optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99, eps=1e-08)</pre> # - alpha: 平滑常数
Adagrad	<pre>optimizer = optim.Adagrad(model.parameters(), lr=0.01, lr_decay=0, weight_decay=0)</pre> # - lr_decay: 学习率衰减 # - weight_decay: L2 正则化项

PyTorch函数用法和介绍——配置训练方法2

2) **损失函数 (Loss Functions)**，用于衡量模型预测与真实标签之间的差异，指导优化过程，常用的损失函数介绍：

损失函数	中文说明	用途	注意事项
CrossEntropyLoss()	交叉熵损失 (Cross Entropy Loss)	用于多分类问题	输入应为原始分数（未经过softmax），目标应为类别索引
BCELoss()	二元交叉熵损失 (Binary Cross Entropy)	用于二分类问题	输入需要经过sigmoid激活
BCEWithLogitsLoss()	带有logits的二元交叉熵损失	用于二分类问题	输入不需要经过sigmoid
NLLLoss()	负对数似然损失 (Negative Log Likelihood Loss)		需要与LogSoftmax配合使用
MSELoss()	均方误差损失 (Mean Squared Error)	用于回归问题	
L1Loss()	平均绝对误差损失 (Mean Absolute Error)	用于回归问题	
SmoothL1Loss()	平滑L1损失 (Smooth L1 Loss)	用于回归问题	对异常值不如MSE敏感

PyTorch函数用法和介绍——训练和评估

➤ 训练模式 `model.train()`

- 训练步骤，通常包括以下步骤：
 - 1) 将模型设置为训练模式
 - 2) 遍历训练数据加载器 (`DataLoader`)
 - 3) 将数据输入模型，得到输出
 - 4) 计算损失 (使用损失函数)
 - 5) 反向传播 (计算梯度)
 - 6) 使用优化器更新模型参数
 - 7) 清零梯度 (防止梯度累积)。

➤ 评估模式 `model.eval()`

- 评估通常包括以下步骤：
 - 1) 将模型设置为评估模式
 - 2) 遍历测试数据加载器 (`DataLoader`)
 - 3) 将数据输入模型，得到输出
 - 4) 计算损失和评估指标 (如准确率)。

注意：在评估时，我们不需要计算梯度，因此使用 `torch.no_grad()` 上下文管理器来禁用梯度计算，以减少内存消耗并加速计算。

目录



实验作业——内容

➤ 任务一：环境搭建与验证

- (1) 使用conda创建虚拟环境dl_lab1;
- (2) 安装PyTorch CPU或者GPU环境，并验证环境是否安装成功;
- (3) 理解示例代码，并调试运行。

➤ 任务二：调试模型

- (1) 设置不同的批量大小(32, 64, 128)，比较模型性能;
- (2) 设置不同的网络深度(3层、5层、7层)，比较模型性能;
- (3) 设置不同的神经元数量(64、128、256、512)，比较模型性能;
- (4) 设置不同的激活函数（如Sigmoid、Tanh、LeakyReLU等），比较模型性能;
- (**提示**：固定其他参数，仅**改变一个变量**，记录**准确率、训练速度、Loss 变化**，用图/表展示)；
- (5) 在实验报告中，请以**图/表**形式 展示调参对比结果。

实验作业——内容

➤ 任务三：掌握反向传播的编程实现，**在全连接神经网络中运行反向传播，并观察效果。**

(1) 按照函数 $y = f(x) = x^6 + 3x^5 + 4x^4 + x + 2$ **生成一组需要拟合的数据**。例如取10000组 x ，其中 x 服从 $[-2, 2]$ 间的均匀分布，按照函数计算对应的 $\bar{y}$ ，得到数据组 $(x_i, y_i | i=1, 2, \dots, 10000)$

(2) **搭建一个全连接神经网络（多层感知机）**，例如有6层，每个隐藏层有200个神经元（对应200*200权重矩阵）的全连接神经网络。在神经网络搭建中**尝试不同的激活函数**，例如ReLU、sigmoid、双曲正切等经典激活函数。

(3) 运用神经网络学习数据组 $(x_i, y_i | i=1, 2, \dots, 10000)$ 的特征，**考虑使用多种损失函数**：均方误差（MSE）/平均绝对值误差（MAE）/交叉熵损失函数等等。

(4) 调用反向传播算法计算梯度，**运用梯度下降的方法学习神经网络的参数**。调试迭代步长（学习率）参数，画出损失函数图，观察损失函数在迭代中变化情况，记录内存/显存占用和计算时间。

实验作业——提交方式

实验报告提交至平台 <https://grader.hitsz.edu.cn/courses>

注意：

- 1、用户名用**统一身份登录**；
- 2、请提交到相应的条目「2026春-深度学习」课程 - 实验一；
- 3、提交截止时间：**下周二 5月19号 晚24点**；
- 4、文件夹&压缩包命名要求：**学号_姓名_深度学习实验一**
- 5、提交内容：实验报告**(.pdf文件)**（**参考实验报告模板**），附录带上代码。